

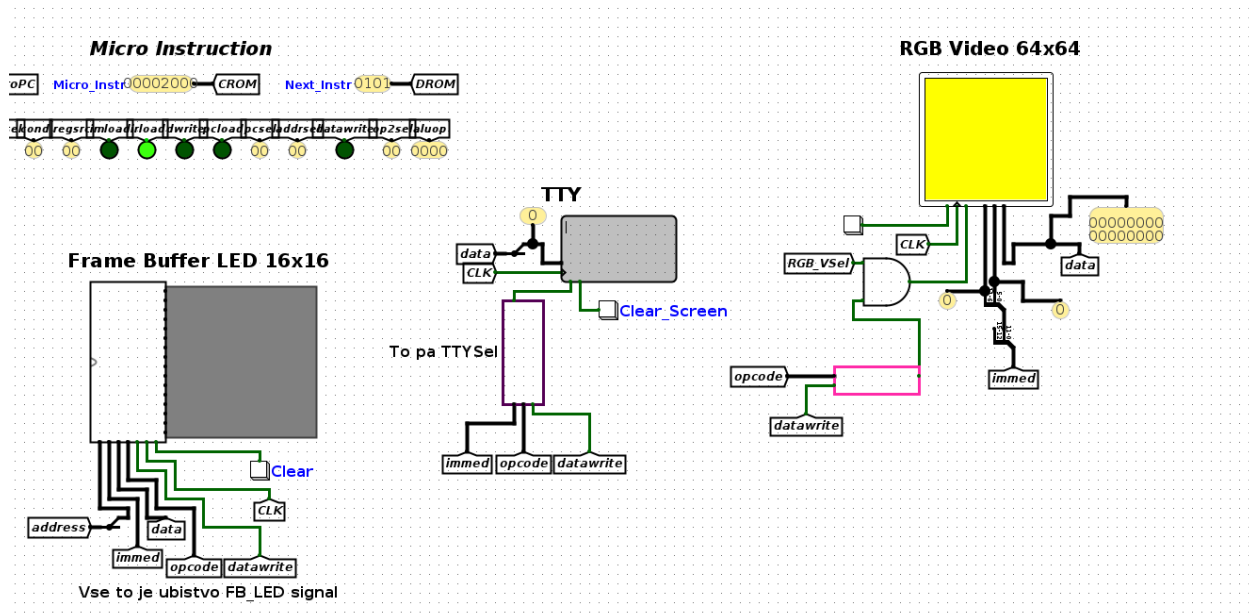
1. domača naloga - MiMo model CPE 2025/26

Mikroprogramirana realizacija strojnih ukazov, testni programi in razširitev V/I prostora

Povzetek

V poročilu je opisana realizacija strojnega ukaza **SW Rd,immed** na nivoju mikroukazov, pregled že realiziranih strojnih ukazov z opkodo, večjo od 15 (aritmetično-logični ukazi s takojšnjim operandom in pogojni skoki), preprost testni program, ki izračuna vsoto zaporednih števil, ter razširitev naslovnega prostora MiMo modela z dodatnima vhodno-izhodnima napravama: **RGB Video 64×64** in TTY, poleg obstoječe naprave **Frame Buffer LED 16×16**.

Vizualni povzetek - celotno vezje z dodanimi napravami



Slika 1: Mikroukazna enota (levo zgoraj) ter povezane V/I naprave – Frame Buffer LED 16×16, TTY in na novo dodan RGB Video 64×64 zaslon.

Naloga 1 - Ukaz SW Rd,immed

Mikroprogramska realizacija (iz datoteke `basic_microcode.def`):

```
# sw Rd,immed      ; Store Rd into M[immed]  Rd -> M[immed]
65:  addrsel=pc  imload=1
      addrsel=immed  datawrite=1  datasel=dreg, goto pcincr
```

Zaporedje mikroukazov in njihov opis:

- **addrsel = pc** - na naslovno vodilo damo vrednost programskega števca PC, da lahko preberemo takojšnji operand (immed), ki sledi opkodi v pomnilniku.
- **imload = 1** - vrednost s podatkovnega vodila (immed) shranimo v takojšnji register (IM register).
- **addrsel = immed** - na naslovno vodilo postavimo vrednost iz takojšnjega registra - to je naslov, kamor bomo pisali.
- **datawrite = 1** - omogočimo pisanje po pomnilniku (oz. po V/I napravi, če naslov pripada njej).
- **datasel = dreg** - na podatkovno vodilo pošljemo vsebino registra Rd (vrednost, ki jo želimo shraniti).
- **goto pcincr** - ker je ukaz porabil dodaten (drugi) 16-bitni register (immed operand), moramo PC dodatno povečati za ena, preden gremo nazaj na fetch naslednjega ukaza.

Ukaz torej porabi **2 mikroukaza** (plus začetni *fetch* in zaključni *pcincr/fetch* skok), kar skupaj z branjem naslednjega ukaza znaša **5 urinih period** (fetch = 2 periodi, izvedba ukaza = 2 periodi, pcincr = 1 perioda).

Naloga 2 - Realizirani strojni ukazi (opkoda > 15)

V datoteki `basic_microcode.def` so poleg osnovnih ALU ukazov (opkode 0–15) realizirani tudi zahtevnejši, netrivialni ukazi z opkodo, večjo od 15, iz različnih skupin: ukazi s takojšnjim operandom (ALU-immediate) in pogojni skoki. Vsi našteti ukazi zahtevajo branje dodatnega 16-bitnega takojšnjega operanda iz pomnilnika (immed), zato je njihova realizacija sestavljena iz več mikroukazov kot pri osnovnih R-R-R ukazih.

Aritmetično-logični ukazi s takojšnjim operandom (skupina immediate, opkode 16-30)

```
#addi Rd,Rs,immed (16)
16: addrsel=pc imload=1
    aluop=add op2sel=immed dwrite=1 regsrc=aluout, goto pcincr

#subi Rd,Rs,immed (17)
17: addrsel=pc imload=1
    aluop=sub op2sel=immed dwrite=1 regsrc=aluout, goto pcincr

... (analogno: muli, divi, remi, andi, ori, xori, nandi, nori,
    lsli, lsri, asri, roli, rori – opkode 18-30)
```

Delovanje (na primeru `addi`): najprej se prebere takojšnji operand (`addrsel=pc`, `imload=1`), nato ALU izračuna vsoto registra `Rs` in takojšnjega operanda (`op2sel=immed`), rezultat se zapiše v `Rd` (`dwrite=1`, `regsrc=aluout`), na koncu pa se PC dodatno poveča (`pcincr`), ker je bil prebran še dodaten 16-bitni operand.

Pogojni skoki s primerjavo dveh registrov (opkode 33-37)

```
#jeq Rs,Rt,immed (33)
33: addrsel=pc imload=1
    aluop=sub op2sel=treg, if z then jump else pcincr

#jne Rs,Rt,immed (34)
34: addrsel=pc imload=1
    aluop=sub op2sel=treg, if z then pcincr else jump
```

`#jgt Rs,Rt,immed (35)` / `#jlt Rs,Rt,immed (37)` - analogno, testira se `n` namesto `z`. Tudi tu se najprej prebere takojšnji naslov skoka, nato ALU izračuna razliko `Rs–Rt` zgolj zaradi nastavljanja zastavic (`z`, `n`), rezultat pa se NE zapiše v register. Glede na stanje zastavice `z` (enakost) oz. `n` (predznak) se mikroprogram razveji: `jump` naloži `PC = immed`, `pcincr` pa izvede navaden nadaljnji korak (skok ni izveden).

Primerjava z ničlo (opkoda 39)

```
#jeqz Rs,immed (39)
39: addrsel=pc imload=1
    aluop=sub op2sel=const0, if z then jump else pcincr
```

Ukaz je funkcionalno podoben zgornjim skokom, le da se register `Rs` primerja s konstanto 0 (`op2sel=const0`) namesto z registrom `Rt` - uporaben je za hitro preverjanje izteka zanke ali pogojnega izhoda.

Naloga 3 - Testni program

Za preizkus zgoraj opisanih ukazov (predvsem `subi`, `jnez`, `sw`) smo napisali preprost testni program, ki izračuna vsoto $5+4+3+2+1$ (šteje navzdol) in rezultat shrani v pomnilnik na naslov 100. Na koncu program preide v neskončno zanko (`halt`).

```
main:  li    r1, 0          # sum = 0
        li    r2, 5          # counter = 5
loop:   add   r1, r1, r2      # sum = sum + counter
        subi  r2, r2, 1      # counter = counter - 1
        jnez  r2, loop       # ce counter != 0, ponovi zanko
        sw    r1, 100        # shrani koncno vsoto v M[100]
halt:   jnez  r1, halt        # neskoncna zanka (r1 != 0 -> vedno skoci nazaj)
```

Opis poteka in trajanje posameznih ukazov

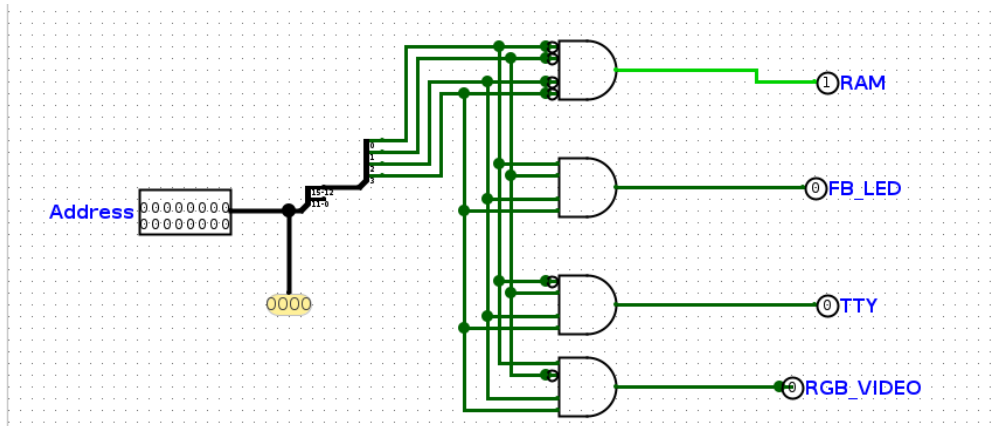
Vsakemu strojnemu ukazu se doda še 2 urini periodi za fazo *fetch* (branje ukaza iz pomnilnika in povečanje PC). V spodnji tabeli je za vsak ukaz podano število urinih period, ki jih porabi njegova mikroprogramska izvedba (brez fetch faze).

Vrstica	Ukaz	Opis delovanja	Cikli (brez fetch)
1	li r1,0	Naloži začetno vrednost 0 v register r1 (sum).	2
2	li r2,5	Naloži začetno vrednost 5 v register r2 (counter).	2
3	add r1,r1,r2	Prišteje trenutni counter k sum (izvede se 5×, enkrat na iteracijo).	1
4	subi r2,r2,1	Zmanjša counter za 1 (izvede se 5×).	3
5	jnez r2,loop	Če counter != 0, skoči nazaj na 'loop' (izvede se 5×, zadnjič pogoj ni izpolnjen).	3
6	sw r1,100	Shrani končno vsoto (15) na naslov 100 v pomnilniku.	3
7	jnez r1,halt	Neskončna zanka - r1=15 je vedno različen od 0, zato program ostane tu (halt).	3

Zanka (ukazi `add`, `subi`, `jnez`) se izvede 5-krat (counter = 5,4,3,2,1). Vsak prehod skozi zanko porabi $(1+3+3) + 3 \times 2$ (fetch) = 13 urinih period, kar za pet ponovitev znese 65 period. Skupno trajanje programa do vstopa v `halt` zanko je torej: uvodna nalaganja $2 \times (2+2)=8$ + zanka 65 + shranjevanje sum-a $(3+2)=5$ + vstop v `halt` $(3+2)=5$, torej **83 urinih period**. Od tu naprej se program ustavi v neskončni zanki na ukazu `jnez r1, halt`.

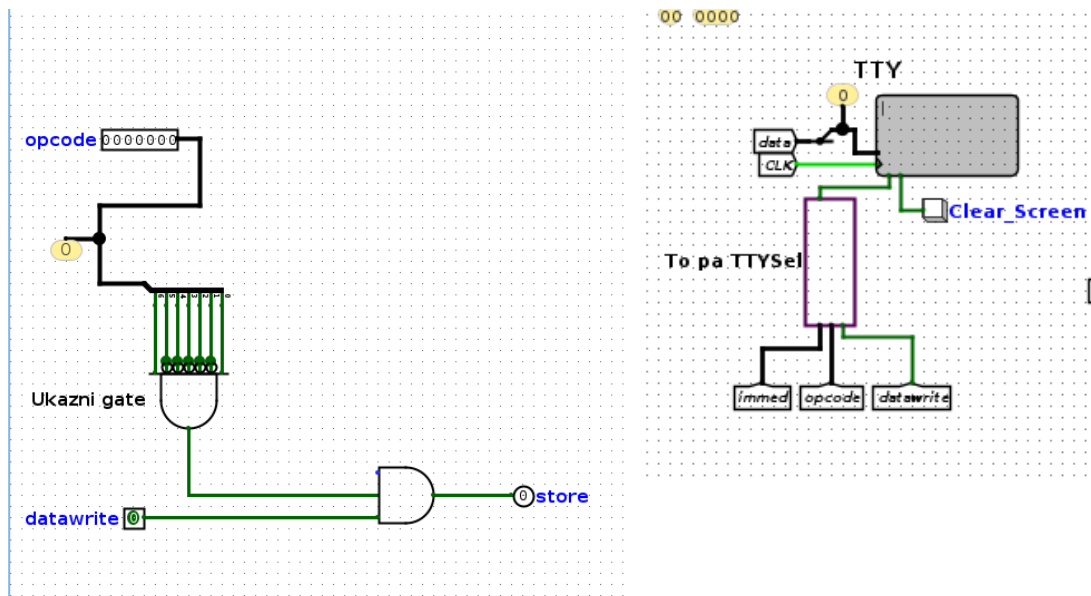
Naloga 4 - Naslovno dekodiranje V/I naprav

Pomnilniški prostor MiMo modela je razdeljen med RAM in tri pomnilniško preslikane V/I naprave (*memory-mapped I/O*): Frame Buffer LED 16×16, TTY in na novo dodan RGB Video 64×64 zaslon. Naslovni dekoder uporablja nepopolno naslovno dekodiranje – primerja le nekaj najvišjih bitov naslova, ne vseh – kar poenostavi vezje, saj naprava zasede cel blok naslovnega prostora.



Slika 2: Naslovni dekoder – iz zgornjih bitov naslova (Address) generira izbirne signale RAM, FB_LED, TTY in RGB_VIDEO. Privzeto (če noben poseben vzorec ni zaznan) je izbran RAM.

Vsaka naprava ima poleg globalnega izbirnega signala iz dekoderja še svoj lokalni "ukazni gate", ki primerja opkodo trenutnega ukaza z ukazom za pisanje (npr. sw) in ga poveže s skupnim signalom datawrite. Šele kadar sta izpolnjena oba pogoja – naslov pripada napravi IN gre za pisalni ukaz – se sproži lokalni store signal naprave.



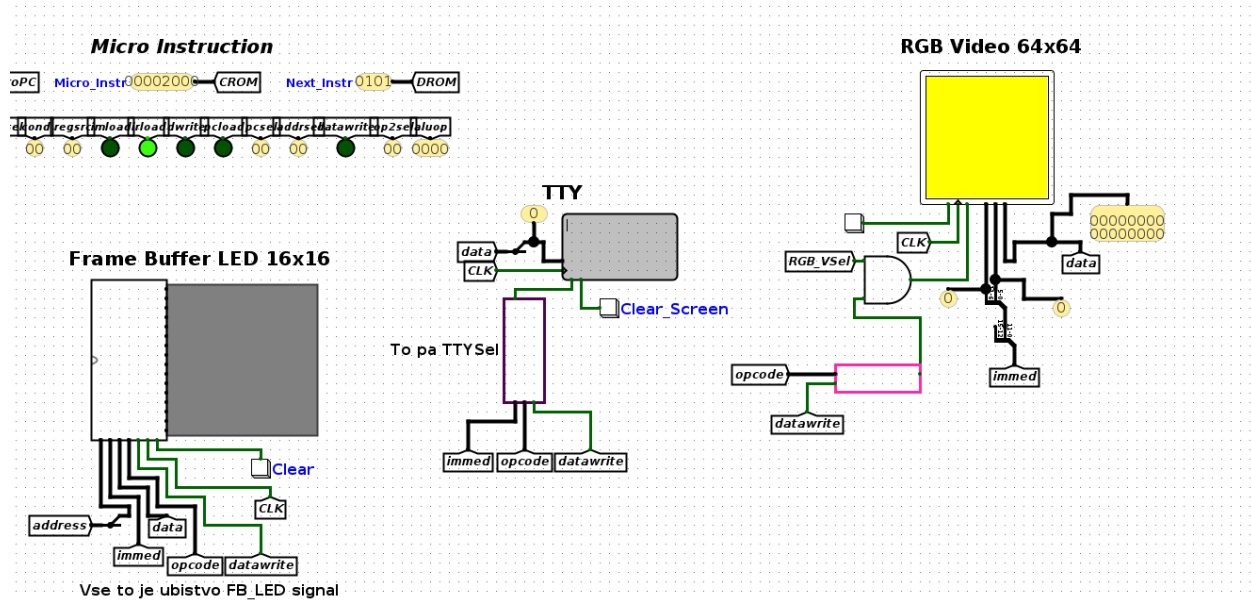
Slika 3: Levo – lokalni "ukazni gate", ki iz opkode in globalnega datawrite signala generira lokalni store signal naprave. Desno – TTY naprava: ob zaznanem zapisu prikaže vrednost iz data, signal Clear_Screen pa počisti izpis.

Na enak način je vključena tudi naprava Frame Buffer LED 16×16 (naslov, podatek in datawrite gredo skozi enak dekodirni mehanizem, dodan je še vhod Clear za brisanje zaslona) ter novo dodani RGB Video 64×64 zaslon (naslednja naloga), kar zagotavlja, da vsaka naprava reagira izključno na zapise znotraj svojega dodeljenega naslovnega obsega, medtem ko preostale naprave tak zapis ignorirajo.

Testni program za V/I napravo (glej Naloga 5, program `test_nightrider.s`) je bil po uvedbi dekodiranja preverjen na dejanski napravi (BIN_LED na naslovu 49152) in se je izvajal pravilno – izrisani vzorec se je pojavljal izključno na dodeljeni napravi, medtem ko RAM in ostale naprave niso bile prizadete.

Naloga 5 - Dodatna vhodno-izhodna naprava: RGB Video 64×64

V pripravljen prostor za dodatno V/I napravo smo vključili barvni video zaslon **RGB Video 64×64**, poleg obstoječe lastne naprave z 16-bitnim registrom (BIN_LED, uporabljena v testnem programu `test_nightrider.s`).



Slika 4: Vezje dodane naprave RGB Video 64×64 (desno) - lokalni ukazni gate primerja opkodo in datawrite, RGB_VSel iz naslovnega dekoderja pa napravo omogoči le, ko CPE naslavlja njen del pomnilniškega prostora. Vhod immed/data nosi barvo/piksel za izris.

Koraki vključitve naprave v MiMo model:

- V naslovnem dekodерju (glej Sliko 2) smo dodali nov izhodni signal `RGB_VIDEO0`, ki se aktivira pri naslovih iz dodeljenega bloka naslovnega prostora, rezerviranega za novo napravo.
- Dodali smo lokalni "ukazni gate", ki iz opkode ukaza in skupnega signala `datawrite` generira lokalni signal za pisanje (analogno kot pri TTY in FB_LED).
- Lokalni signal smo z vrati AND združili s signalom `RGB_VSel` iz naslovnega dekoderja - naprava tako piše samo, kadar naslov dejansko pripada njej.
- Podatkovna pot naprave sprejema `immed/data` kot vrednost barve posameznega piksla, `opcode` pa določa naslov piksla znotraj 64×64 mreže (analogno naslavljanju pri FB_LED, le v večji resoluciji).
- Napravo poganja isti CLK kot ostale komponente, dodan je še vhod za brisanje zaslona (`Clear/Clear_Screen`), tako da se lahko pred izpisom novega vzorca zaslon počisti.
- Delovanje je bilo preizkušeno s testnim programom, ki po vrsti zapisuje barvne vrednosti na posamezne naslove znotraj naslovnega prostora naprave in preveri, da se izpis pojavi izključno na RGB Video napravi, medtem ko RAM, TTY in FB_LED ostanejo nespremenjeni.

Poleg RGB Video naprave je bila z ločenim testnim programom `test_nightrider.s` (Nightrider vzorec - ena prižgana LED potuje po nizu diod v obe smeri) preizkušena tudi lastna izhodna naprava z 16-bitnim registrom (BIN_LED na naslovu 49152).